Fundamentals of python language

INTRODUCTION TO PYTHON THEORY:

INTRODUCTION TO PYTHON AND ITS FEATURES (SIMPLE, HIGH-LEVEL, INTERPRETED LANGUAGE).

-> Python is often described as the "Swiss Army Knife" of programming languages. Created by Guido van Rossum and released in 1991, it was designed with a core philosophy in mind: code readability matters. Today, it is the go-to language for data science, artificial intelligence, web development, and automation because it allows you to do more with less code.

1. It’s Simple

Python uses plain English words. In other languages, you might need 5 or 10 lines of code just to say "Hello." In Python, you only need one. It’s designed so you spend more time solving problems and less time worrying about where to put a semicolon.

2. It’s "High-Level"

This is a fancy way of saying Python does the heavy lifting for you. You don’t need to understand how a computer’s brain (CPU) or memory works to use it. You just give the command, and Python handles the technical "behind-the-scenes" stuff.

3. It’s an "Interpreted" Language

Python reads your code like a human reads a book: one line at a time, from top to bottom. * The Plus Side: If there’s a mistake on line 5, Python stops right there and tells you. You don't have to wait for the whole program to finish "preparing" before you can test it.

4. It’s Multi-Purpose

Python isn't just for one thing. It’s used for:

- Artificial Intelligence
- Building Websites
- Automating Tasks

HISTORY AND EVOLUTION OF PYTHON

-> Python was created by **Guido van Rossum** in the late 1980s at the National Research Institute for Mathematics and Computer Science in the Netherlands.

- **The Origin:** Guido wanted a language that was as easy to read as plain English. He named it after the BBC comedy show *Monty Python's Flying Circus*, not the snake!
- **Python 1.0 (1994):** Introduced basic features like functions and data types.
- **Python 2.0 (2000):** Added "List Comprehensions" and a garbage collector (to clean up computer memory automatically).
- **Python 3.0 (2008):** A major "clean-up" of the language. It wasn't fully compatible with version 2, but it made the language much more consistent and powerful. Today, Python 3 is the standard.

ADVANTAGES OVER OTHER LANGUAGES

-> The advantages of python over another languages:

- Write Less, Do More: You can often accomplish in 3 lines of Python what takes 10–20 lines in Java.
- Easy to Read: It uses "indentation" (spacing) to organize code, which makes it look very clean.
- Huge Library: It has a "plugin" for everything. Want to do AI? Use TensorFlow. Want to analyse data? Use Pandas.
- Free and Open: It's free to use, and a massive community of millions of people helps fix bugs and add new features.

SETTING UP YOUR ENVIRONMENT

-> To write Python, you need an Interpreter (the engine) and an Editor (where you type). Here are the three most popular setups:

A. VS Code (Recommended for Beginners)

- **What it is:** A lightweight, fast, and very popular editor from Microsoft.
- **Best for:** General programming and web development.
- **Setup:** Download VS Code, then install the "Python" extension from the marketplace.

B. PyCharm

- **What it is:** A "Full IDE" (Integrated Development Environment) specifically for Python.
- **Best for:** Large, professional projects. It has very smart error checking.

C. Anaconda

- **What it is:** A "bundle" that includes Python and hundreds of tools for Data Science.
- **Best for:** Math, science, and data analysis. It includes **Jupyter Notebooks**, which let you run code in small chunks.

WRITING AND EXECUTING YOUR FIRST PYHTON PROGRAM

-> print ("Hello, I am a Python programmer!")

How to Execute It:

1. **Open your Editor:** (VS Code or PyCharm).
2. **Create a New File:** Save it as hello.py (the .py tells the computer it's a Python file).
3. **Type the code:** Copy the line above.
4. **Run it:** In **VS Code**, click the little "Play" button in the top right corner.
    - In a **Terminal/Command Prompt**, type: python hello.py and hit Enter.

PROGRAMMING STYLE:

UNDERSTANDING PYTHON'S PEP 8 GUIDELINES

-> PEP 8 is the "Constitution" of Python. It was written in 2001 by Guido van Rossum and others to ensure that Python code remains consistent, aesthetic, and professional. The goal is simple: Readability counts.

1. The Layout (How it looks)

- **Indentation:** Use 4 spaces per level. Never use tabs.
- **Line Length:** Keep lines under 79 characters.
- **Blank Lines:** 2 blank lines between functions/classes.

2. Naming Conventions (The "Look" of Code)

Naming helps you distinguish between a constant, a variable, and a class just by looking at the casing.

3. Whitespace (Spacing)

- **Operators:** Put one space around = and + (e.g., x = 5 + 2).
- **Brackets:** No spaces inside (e.g., print(x, y), not print(x, y)).
- **Commas:** Space **after** the comma, not before.

4. Best Practices

- **Imports**: Always at the very top of the file.

- **Booleans**: Use if is_active: instead of if is_active == True:.
- **Comments**: Use # to explain why you wrote the code, not what the code is doing.

INDENTATION, COMMENTS, AND NAMING CONVENTIONS IN PYTHON

1. **Indentation**
   - **The Standard:** Use **4 spaces** per indent level.
   - **Why?** Python uses indentation to define "blocks" of code (like what sits inside a loop or a function). Incorrect indentation will cause your program to crash.
2. **Naming Conventions**

Using the right "case" helps other coders identify what a name represents at a glance:

- **Variables & Functions:** Use snake_case (e.g., user_score, calculate_total).
- **Classes:** Use PascalCase (e.g., SmartPhone, UserAccount).
- **Constants:** Use UPPER_CASE (e.g., PI = 3.14, MAX_USERS = 100).

3. **Comments**
   - Use # for short notes.
   - **Pro Tip:** Don't comment on *what* the code does (the code should be clear); comment on **why** you wrote it that way if the logic is tricky.

WRITING READABLE AND MAINTAINABLE CODE

- To make your code "clean," follow these three simple habits:
- Be Descriptive: Use names like days_until_expiry instead of just d.
- One Task, One Function: Don't make a single function do ten different things. Break them up.
- Use Whitespace: Use blank lines to separate logic sections. Just like a book has paragraphs, your code needs breathing room.

CORE PYTHON CONCEPTS:

UNDERSTANDING DATA TYPES: INTEGERS, FLOATS, STRINGS, LISTS, TUPLES, DICTIONARIES, SETS

-> Python is **dynamically typed**, meaning you don't need to tell Python what the data type is; it figures it out automatically.

Category	Type	Description	Example
Numeric	`int`	Whole numbers (positive or negative)	`10, -5`
	`float`	Numbers with decimals	`10.5, 3.14`
Text	`str`	Text inside quotes	`"Hello"`
Sequence	`list`	Ordered, changeable list	`[1, 2, "apple"]`
	`tuple`	Ordered, **unchangeable** list	`(1, 2, 3)`
Mapping	`dict`	Key-value pairs (like a real dictionary)	`{"name": "Alice"}`
Set	`set`	Unordered collection of **unique** items	`{1, 2, 3}`

PYTHON VARIABLES AND MEMORY ALLOCATION

-> A **variable** is a name that refers to a value.

- **The Pointer System:** In Python, variables don't *contain* the data. Instead, they act like **labels** or pointers. The data is stored in a specific spot in your computer's memory, and the variable just points to that location.
- **Dynamic Allocation:** If you change x = 5 to x = "Hello", Python automatically wipes the number 5 from memory (Garbage Collection) and points x to the new text.

PYTHON OPERATORS: ARITHMETIC, COMPARISON, LOGICAL, BITWISE

-> Operators are symbols that perform operations on your data.

Arithmetic (Math)

Used for calculations:

- + (Add), - (Subtract), * (Multiply), / (Divide)
- % (Remainder/Modulus): 10 % 3 results in 1.
- ** (Exponent/Power): 2 ** 3 is 8.

Comparison (Checking)

Result is always True or False:

- == (Equal to), != (Not equal to)
- >, <, >=, <=

Logical (Combining)

Used to combine multiple conditions:

- and: True if **both** are true.
- or: True if **at least one** is true.
- not: Flips the result (not True becomes False).

Bitwise (Binary)

These operate on the **bits (0s and 1s)** of numbers. These are rarely used by beginners but common in hardware programming or cryptography:

- & (AND), | (OR), ^ (XOR), ~ (NOT), << (Left Shift).

CONDITIONAL STATEMENTS:

INTRODUCTION TO CONDITIONAL STATEMENTS: IF, ELSE, ELIF

-> Conditional statements are the "decision-makers" of your code. They allow your program to perform different actions based on whether a specific condition is **True** or **False**.

1. The if Statement

The if statement is the simplest form. It executes a block of code **only if** the condition is met.

Example:

```
age = 20
if age >= 18:
    print("You are an adult.")
```

- **The Colon (:):** This tells Python that a block of code is starting.
- **Indentation:** The print line is indented (4 spaces). This tells Python it belongs *inside* the if statement.

2. The else Statement

The else statement provides an "alternative" plan. It runs only if the if condition was False.

Example:

```
password = "123"
if password == "12345":
    print("Access Granted")
else:
    print("Access Denied")
```

3. The elif Statement (Short for "Else If")

What if you have more than two options? elif allows you to check multiple conditions in a specific order. Python checks them one by one and stops as soon as it finds a True one.

Example:

```
score = 85

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

4. Nested Conditionals

You can even put an if statement *inside* another if statement. This is useful for complex logic.

Example:

```
is_weekend = True
is_sunny = True

if is_weekend:
    if is_sunny:
        print("Go to the beach!")
    else:
        print("Watch a movie at home.")
else:
    print("Go to work.")
```

LOOPING (FOR & WHILE):

INTRODUCTION TO **FOR** AND **WHILE** LOOPS

-> In Python, loops are the tools we use to repeat a block of code multiple times. Instead of writing the same line over and over, you tell Python to "loop" through the instructions.

There are two main types of loops: FOR loop and WHILE loop.

1. The for Loop (Definite Iteration)

A for loop is used when you know in advance how many times you want to run a block of code, or when you want to go through every item in a collection (like a list, a string, or a range of numbers).

How it works:

- It "iterates" over a sequence.
- It automatically stops when it reaches the end of that sequence.

Example:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

2. The while Loop (Indefinite Iteration)

A while loop is used when you want to repeat a block of code as long as a certain **condition** remains true. You use this when you don't necessarily know how many laps the loop will take.

How it works:

- It checks the condition **before** every lap.
- If the condition is True, it runs.
- If the condition is False, it stops immediately.

Example:

```
count = 1
while count <= 3:
    print(count)
    count += 1  # This increases count so the loop eventually stops
```

HOW LOOPS WORK IN PYTHON

Every loop in Python follows a four-step lifecycle. If any of these steps are missing or broken, the loop will either not run at all or run forever.

1. **Initialization:** The loop identifies its starting point (e.g., the first item in a list or a counter set to 0).
2. **Condition Check:** Before running the code, Python asks: *"Is the condition still True?"* or *"Is there another item left in the sequence?"*
3. **Execution (The Body):** If the answer is **Yes**, Python executes the indented block of code.
4. **Iteration/Update:** Python move to the next item (in for loops) or updates the control variable (in while loops) and jumps back to Step 2.

USING LOOPS WITH COLLECTIONS (LISTS, TUPLES, ETC.)

Python is uniquely "iterator-friendly," meaning it's for loop is designed to automatically traverse data structures without the need for manual indexing.

- Iterating Over Lists:

A **List** is an ordered, mutable collection. When used with a for loop, Python assigns each element to the loop variable in the order they appear (from index 0 to n-1).

```
numbers =
for num in numbers:
    print(num)
```

Output:

[1, 2, 3]

1

2

3

- Iterating Over Tuples

A **Tuple** is an ordered, immutable collection. Iteration works identically to lists, but tuples are often used for data that should not be modified during the program's execution.

```
colors = ("red", "green", "blue")
for color in colors:
    print(color)
```

Output:

red

green

blue

- Iterating Over Strings

In Python, a **String** is considered a sequence of individual characters. A loop will "unpack" the string one character at a time.

```
name = "Python"
for letter in name:
    print(letter)
```

Output:

P

y

y

h

o

n

- Iterating Over Dictionaries

A **Dictionary** stores data in `Key: Value` pairs. By default, iterating over a dictionary only returns the **Keys**. To access the data (Values), you must reference the key within the loop.

```
student = {"name": "John", "age": 20}
for key in student:
    print(key, student[key])
```

Output:

name: John

age: 20

CONTROL STATEMENTS (BREAK, CONTINUE, AND PASS):

Understanding the role of break, continue, and pass in Python loops

In Python, these three keywords—**break**, **continue**, and **pass**—are known as **Loop Control Statements**. They allow you to deviate from the standard "top-to-bottom" flow of a loop based on specific conditions.

- The **break** statement

The break statement is used to terminate the loop entirely. When a break is encountered, the program immediately jumps to the code following the loop.

Example:

```
numbers =

for n in numbers:
    if n == 30:
        print("Found 30! Exiting loop.")
        break
    print(f"Checking: {n}")

print("Loop finished.")
```

Output:

```
Checking: 10
Checking: 20
Found 30! Exiting loop.
Loop finished.
```

- The **continue** statement

The continue statement skips the remaining code inside the loop for the **current iteration** only and moves to the next item in the sequence.

Example:

```
for i in range(1, 6):
    if i % 2 == 0:
        continue  # Skip the print statement for even numbers
    print(f"Odd number: {i}")
```

Output:

```
Odd number: 1
Odd number: 3
Odd number: 5
```

- The **pass** statement

The pass statement is a null operation; nothing happens when it executes. It is used as a placeholder when a statement is syntactically required but no action is needed yet.

Example:

```python
colors = ["red", "blue", "green"]

for color in colors:
    if color == "blue":
        pass  # TODO: Handle blue color logic later
    else:
        print(f"Processing {color}")
```

Output:

```
Processing red
Processing green
```